Лабораторная работа №20

Тема работы: «Создание и обработка главного и контекстного меню. Разработка алгоритмов и программ решения различных задач с использованием библиотечных модулей пользователя, внешних файлов с использованием дополнительных элементов управления диалоговым окном»

1 Цель работы

Сформировать умения разрабатывать алгоритмы и программы решения различных задач с использованием библиотечных модулей пользователя, главного и контекстного меню.

2 Задание

2.1. В разработанном в лабораторной работе № 19 Калькулятор, добавить «Меню», содержащее информацию о приложении и авторе разработки (рисунок 1).

2.2. Организовать ввод данных Калькулятора с помощью клавиатуры.

2.3. Функцию обработки нажатий на клавиши клавиатуры оформить в библиотеку .dll.

3 Оснащение

Задание по варианту, ЭВМ, среда разработки Visual Studio.

4 Порядок выполнения работы

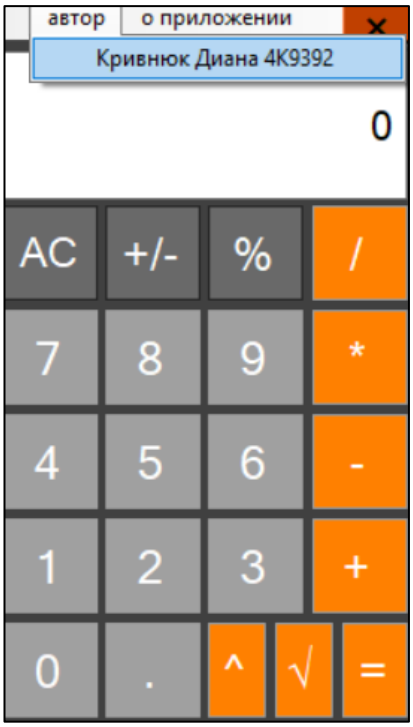

Рисунок 1 - Меню в калькуляторе

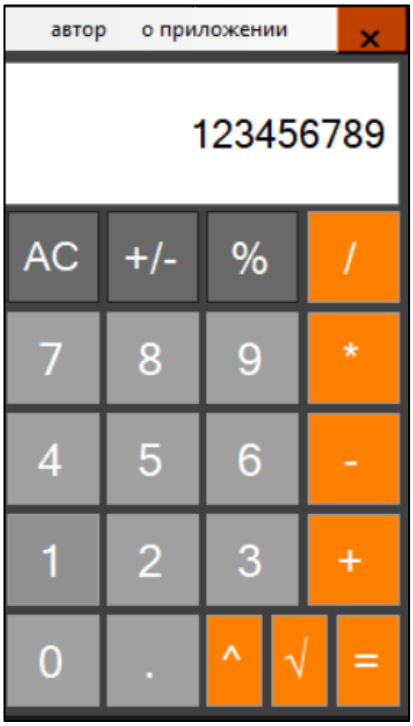

Рисунок 2 - Ввод данных с клавиатуры

Код программы:

```
1. #pragma once
2.
3. namespace Project3 {
4.
5.     using namespace System;
6.     using namespace System::ComponentModel;
7.     using namespace System::Collections;
8.     using namespace System::Windows::Forms;
9.     using namespace System::Data;
10.    using namespace System::Drawing;
11.    using namespace System::Runtime::InteropServices;
12.
13.
14.
15.    /// <summary>
16.    /// Сводка для MyForm
17.    /// </summary>
18.    ///
19.    ///
20.
21.    public ref class MyForm : public System::Windows::Forms::Form
22.
23.    {
24.    public:
25.            MyForm(void)
26.            {
27.                    InitializeComponent();
28.                    this->KeyPreview = true;
29.                    this->KeyDown += gcnew System::Windows::Forms::KeyEventHandler(this,
&MyForm::MyForm_KeyDown);
30.            }
31.
32.    protected:
33.
34.            /// <summary>
35.            /// Освободить все используемые ресурсы.
36.            /// </summary>
37.            ~MyForm()
38.            {
39.                    if (components)
40.                    {
```

```
41.                                 delete components;
42.                         }
43.             }
44.
45.
46.     private:
47.             [DllImport("projec3t.dll", CallingConvention = CallingConvention::StdCall, CharSet =
CharSet::Ansi)]
48.                         static IntPtr ProcessKey(int key);
49.
50.     private: System::Windows::Forms::Button^ button1;
51.     private: System::Windows::Forms::Button^ button2;
52.     private: System::Windows::Forms::Button^ button3;
53.     private: System::Windows::Forms::Button^ button4;
54.     private: System::Windows::Forms::Button^ button5;
55.     private: System::Windows::Forms::Button^ button6;
56.     private: System::Windows::Forms::Button^ button7;
57.     private: System::Windows::Forms::Button^ button8;
58.     private: System::Windows::Forms::Button^ button9;
59.     private: System::Windows::Forms::Button^ button10;
60.     private: System::Windows::Forms::Button^ button11;
61.     private: System::Windows::Forms::Button^ button12;
62.
63.
64.     private: System::Windows::Forms::Button^ button15;
65.     private: System::Windows::Forms::Button^ button16;
66.     private: System::Windows::Forms::Button^ button17;
67.     private: System::Windows::Forms::Button^ button18;
68.     private: System::Windows::Forms::Button^ button19;
69.
70.     private: System::Windows::Forms::Button^ button21;
71.     private: System::Windows::Forms::Button^ button13;
72.     private: System::Windows::Forms::Button^ button14;
73.
74.     private: System::Windows::Forms::Button^ button23;
75.     private: System::Windows::Forms::Button^ button20;
76.     private: System::Windows::Forms::Label^ label1;
77.
78.
79.             double firstOperand;
80.             System::String^ currentOperation;
81.             bool isNewOperation;
82.             bool hasError;
83.
84.
85.     private: System::Windows::Forms::MenuStrip^ menuStrip1;
86.
87.     private: System::Windows::Forms::ToolStripMenuItem^ авторToolStripMenuItem;
88.     private: System::Windows::Forms::ToolStripMenuItem^ прил;
89.     private: System::Windows::Forms::ToolStripMenuItem^ оПриложенииToolStripMenuItem;
90.     private: System::Windows::Forms::ToolStripMenuItem^ кривнюкДиана4К9392ToolStripMenuItem;
91.     private: System::Windows::Forms::ToolStripMenuItem^ приложениекалькуляторToolStripMenuItem;
92.     private: System::Windows::Forms::Button^ button22;
93.     private: System::Windows::Forms::Button^ button24;
94.
95.
96.     private: System::ComponentModel::IContainer^ components;
97.
98.                 /// <summary>
99.             /// Обязательная переменная конструктора.
100.            /// </summary>
101.
102.
103. //…design
628.
629. #pragma endregion
630.            System::Void MyForm_KeyDown(System::Object^ sender, System::Windows::Forms::KeyEven-
tArgs^ e)
631.            {
632.
633.                        bool shiftPressed = (Control::ModifierKeys & Keys::Shift) == Keys::Shift;
634.
635.
636.                        if (e->KeyCode >= Keys::D0 && e->KeyCode <= Keys::D9 && !shiftPressed) {
637.                                int num = (int)e->KeyCode - (int)Keys::D0;
638.                                PressDigit(num);
```

```
639.                                    e->SuppressKeyPress = true;
640.                                    e->Handled = true;
641.                                    return;
642.                            }
643.
644.                            if (e->KeyCode >= Keys::NumPad0 && e->KeyCode <= Keys::NumPad9) {
645.                                    int num = (int)e->KeyCode - (int)Keys::NumPad0;
646.                                    PressDigit(num);
647.                                    e->SuppressKeyPress = true;
648.                                    e->Handled = true;
649.                                    return;
650.                            }
651.
652.
653.                            if (e->KeyCode == Keys::D8 && shiftPressed) {
654.                                    button16->PerformClick();
655.                                    e->SuppressKeyPress = true;
656.                                    e->Handled = true;
657.                                    return;
658.                            }
659.
660.
661.                            if (e->KeyCode == Keys::Multiply) {
662.                                    button16->PerformClick();
663.                                    e->SuppressKeyPress = true;
664.                                    e->Handled = true;
665.                                    return;
666.                            }
667.
668.
669.                            if (e->KeyCode == Keys::Divide || e->KeyCode == Keys::Oem2) {
670.                                    button15->PerformClick();
671.                                    e->SuppressKeyPress = true;
672.                                    e->Handled = true;
673.                                    return;
674.                            }
675.
676.
677.                            if (e->KeyCode == Keys::D6 && shiftPressed) {
678.                                    button19->PerformClick();
679.                                    e->SuppressKeyPress = true;
680.                                    e->Handled = true;
681.                                    return;
682.                            }
683.
684.
685.                            if (e->KeyCode == Keys::Add || (e->KeyCode == Keys::Oemplus && shiftPressed)) {
686.                                    button18->PerformClick();
687.                                    e->SuppressKeyPress = true;
688.                                    e->Handled = true;
689.                                    return;
690.                            }
691.
692.
693.                            if (e->KeyCode == Keys::Subtract || e->KeyCode == Keys::OemMinus) {
694.                                    button17->PerformClick();
695.                                    e->SuppressKeyPress = true;
696.                                    e->Handled = true;
697.                                    return;
698.                            }
699.
700.
701.                            if (e->KeyCode == Keys::Enter || e->KeyCode == Keys::Return) {
702.                                    button21->PerformClick();
703.                                    e->SuppressKeyPress = true;
704.                                    e->Handled = true;
705.                                    return;
706.                            }
707.
708.                            if (e->KeyCode == Keys::Oemplus && !shiftPressed) {
709.                                    button21->PerformClick();
710.                                    e->SuppressKeyPress = true;
711.                                    e->Handled = true;
712.                                    return;
713.                            }
714.
```

```
715.
716.                        if (e->KeyCode == Keys::Decimal || e->KeyCode == Keys::OemPeriod) {
717.                                button11->PerformClick();
718.                                e->SuppressKeyPress = true;
719.                                e->Handled = true;
720.                                return;
721.                        }
722.
723.
724.                        if (e->KeyCode == Keys::R) {
725.                                button20->PerformClick();
726.                                e->SuppressKeyPress = true;
727.                                e->Handled = true;
728.                                return;
729.                        }
730.
731.
732.                        if (e->KeyCode == Keys::Back) {
733.                                String^ text = this->label1->Text;
734.                                if (text->Length > 0) {
735.                                        if (text == "0" || text == "Error" || text->StartsWith("Er-
ror") ||
736.                                                text->StartsWith("Автор") || text->Starts-
With("Оконное")) {
737.                                                this->label1->Text = "0";
738.                                        }
739.                                        else {
740.                                                this->label1->Text = text->Substring(0, text-
>Length - 1);
741.                                                if (this->label1->Text == "" || this->label1-
>Text == "-")
742.                                                        this->label1->Text = "0";
743.                                        }
744.                                }
745.                                isNewOperation = false;
746.                                e->SuppressKeyPress = true;
747.                                e->Handled = true;
748.                                return;
749.                        }
750.
751.
752.                        if (e->KeyCode == Keys::Escape) {
753.                                button12->PerformClick();
754.                                e->SuppressKeyPress = true;
755.                                e->Handled = true;
756.                                return;
757.                        }
758.
759.
760.                        if (e->KeyCode == Keys::F1) {
761.                                button24->PerformClick();
762.                                e->SuppressKeyPress = true;
763.                                e->Handled = true;
764.                                return;
765.                        }
766.
767.
768.                        if (e->KeyCode == Keys::F2) {
769.                                this->label1->Text = "0";
770.                                e->SuppressKeyPress = true;
771.                                e->Handled = true;
772.                                return;
773.                        }
774.                }
775.                private: void PressDigit(int num)
776.                {
777.                        switch (num) {
778.                        case 0: button10->PerformClick(); break;
779.                        case 1: button7->PerformClick(); break;
780.                        case 2: button8->PerformClick(); break;
781.                        case 3: button9->PerformClick(); break;
782.                        case 4: button4->PerformClick(); break;
783.                        case 5: button5->PerformClick(); break;
784.                        case 6: button6->PerformClick(); break;
785.                        case 7: button1->PerformClick(); break;
786.                        case 8: button2->PerformClick(); break;
```

```
787.                        case 9: button3->PerformClick(); break;
788.                        }
789.                }
790.    private: double SafeConvert(String^ text)
791.    {
792.                try {
793.                        text = text->Trim();
794.
795.                        if (text == "" || text == "-" || text == "-.") {
796.                                return 0;
797.                        }
798.
799.                        text = text->Replace(",", ".");
800.
801.                        if (text == "." || text == "-.") {
802.                                return 0;
803.                        }
804.
805.                        return Double::Parse(text,
806.                                System::Globalization::CultureInfo::InvariantCulture);
807.                }
808.                catch (Exception^) {
809.                        return 0;
810.                }
811.    }
812.
813. private: System::Void MyForm_Load(System::Object^ sender, System::EventArgs^ e) {
814.    firstOperand = 0;
815.    currentOperation = "";
816.    isNewOperation = true;
817.    hasError = false;
818.    this->label1->Text = "0";
819.
820.    this->label1->AutoSize = false;
821.    this->label1->Width = this->ClientSize.Width - 5;
822.    this->label1->Height = 80;
823.    this->label1->Font = gcnew System::Drawing::Font("Microsoft Sans Serif", 16);
824.    this->label1->TextAlign = ContentAlignment::MiddleRight;
825.    this->label1->BackColor = Color::White;
826.    this->label1->BorderStyle = Windows::Forms::BorderStyle::FixedSingle;
827.    this->label1->Location = System::Drawing::Point(0, 30);
828.
829.
830.    this->KeyPreview = true;
831.    this->Focus();
832. }
833.
834.
835.
836. private: System::Void button10_Click(System::Object^ sender, System::EventArgs^ e) {
837.
838.    if (this->label1->Text == "Error") {
839.                this->label1->Text = "0";
840.                isNewOperation = false;
841.                hasError = false;
842.    }
843.
844.    else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
845.                this->label1->Text = "0";
846.                isNewOperation = false;
847.    }
848.
849.    else {
850.                this->label1->Text += "0";
851.    }
852. }
853.
854.
855. private: System::Void button7_Click(System::Object^ sender, System::EventArgs^ e) {
856.
857.    if (this->label1->Text == "Error") {
858.                this->label1->Text = "1";
859.                isNewOperation = false;
860.                hasError = false;
861.    }
862.
```

```
863.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
864.             this->label1->Text = "1";
865.             isNewOperation = false;
866.     }
867.     else {
868.             this->label1->Text += "1";
869.     }
870. }
871.
872.
873. private: System::Void button8_Click(System::Object^ sender, System::EventArgs^ e) {
874.     if (this->label1->Text == "Error") {
875.             this->label1->Text = "2";
876.             isNewOperation = false;
877.             hasError = false;
878.     }
879.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
880.             this->label1->Text = "2";
881.             isNewOperation = false;
882.     }
883.     else {
884.             this->label1->Text += "2";
885.     }
886. }
887.
888.
889. private: System::Void button9_Click(System::Object^ sender, System::EventArgs^ e) {
890.     if (this->label1->Text == "Error") {
891.             this->label1->Text = "3";
892.             isNewOperation = false;
893.             hasError = false;
894.     }
895.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
896.             this->label1->Text = "3";
897.             isNewOperation = false;
898.     }
899.     else {
900.             this->label1->Text += "3";
901.     }
902. }
903.
904.
905. private: System::Void button4_Click(System::Object^ sender, System::EventArgs^ e) {
906.     if (this->label1->Text == "Error") {
907.             this->label1->Text = "4";
908.             isNewOperation = false;
909.             hasError = false;
910.     }
911.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
912.             this->label1->Text = "4";
913.             isNewOperation = false;
914.     }
915.     else {
916.             this->label1->Text += "4";
917.     }
918. }
919.
920.
921. private: System::Void button5_Click(System::Object^ sender, System::EventArgs^ e) {
922.     if (this->label1->Text == "Error") {
923.             this->label1->Text = "5";
924.             isNewOperation = false;
925.             hasError = false;
926.     }
927.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
928.             this->label1->Text = "5";
929.             isNewOperation = false;
930.     }
931.     else {
932.             this->label1->Text += "5";
933.     }
934. }
935.
936.
937. private: System::Void button6_Click(System::Object^ sender, System::EventArgs^ e) {
938.     if (this->label1->Text == "Error") {
```

```
939.                this->label1->Text = "6";
940.                isNewOperation = false;
941.                hasError = false;
942.     }
943.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
944.                this->label1->Text = "6";
945.                isNewOperation = false;
946.     }
947.     else {
948.                this->label1->Text += "6";
949.     }
950. }
951.
952.
953. private: System::Void button1_Click(System::Object^ sender, System::EventArgs^ e) {
954.     if (this->label1->Text == "Error") {
955.                this->label1->Text = "7";
956.                isNewOperation = false;
957.                hasError = false;
958.     }
959.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
960.                this->label1->Text = "7";
961.                isNewOperation = false;
962.     }
963.     else {
964.                this->label1->Text += "7";
965.     }
966. }
967.
968.
969. private: System::Void button2_Click(System::Object^ sender, System::EventArgs^ e) {
970.     if (this->label1->Text == "Error") {
971.                this->label1->Text = "8";
972.                isNewOperation = false;
973.                hasError = false;
974.     }
975.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
976.                this->label1->Text = "8";
977.                isNewOperation = false;
978.     }
979.     else {
980.                this->label1->Text += "8";
981.     }
982. }
983.
984.
985. private: System::Void button3_Click(System::Object^ sender, System::EventArgs^ e) {
986.     if (this->label1->Text == "Error") {
987.                this->label1->Text = "9";
988.                isNewOperation = false;
989.                hasError = false;
990.     }
991.     else if (isNewOperation || this->label1->Text == "" || this->label1->Text == "0") {
992.                this->label1->Text = "9";
993.                isNewOperation = false;
994.     }
995.     else {
996.                this->label1->Text += "9";
997.     }
998. }
999.
1000.
1001. private: System::Void button11_Click(System::Object^ sender, System::EventArgs^ e) {
1002.
1003.     if (this->label1->Text == "Error") {
1004.                this->label1->Text = "0,";
1005.                isNewOperation = false;
1006.                hasError = false;
1007.                return;
1008.     }
1009.
1010.
1011.     String^ currentText = this->label1->Text;
1012.
1013.
1014.     if (isNewOperation || currentText == "" || currentText == "-") {
```

```
1015.                this->label1->Text = "0,";
1016.                isNewOperation = false;
1017.                return;
1018.     }
1019.
1020.
1021.     else if (currentText == "0" || currentText == "-0") {
1022.                this->label1->Text = currentText + ",";
1023.                isNewOperation = false;
1024.                return;
1025.     }
1026.
1027.
1028.     bool hasDecimalSeparator = currentText->Contains(",") || currentText->Contains(".");
1029.
1030.
1031.     if (!hasDecimalSeparator) {
1032.                this->label1->Text = currentText + ",";
1033.     }
1034.
1035. }
1036.
1037.
1038.
1039.
1040. private: System::Void button23_Click(System::Object^ sender, System::EventArgs^ e) {
1041.     this->Close();
1042. }
1043. private: System::Void button12_Click(System::Object^ sender, System::EventArgs^ e) {
1044.     this->label1->Text = "0";
1045.     firstOperand = 0;
1046.     currentOperation = "";
1047.     isNewOperation = true;
1048.     hasError = false;
1049. }
1050. private: System::Void button13_Click(System::Object^ sender, System::EventArgs^ e) {
1051.     try {
1052.
1053.                String^ currentText = this->label1->Text;
1054.
1055.
1056.                if (currentText == "" || currentText == "0") {
1057.                          this->label1->Text = "-0";
1058.                          isNewOperation = false;
1059.                }
1060.
1061.                else if (currentText->StartsWith("-")) {
1062.
1063.                          if (currentText == "-0") {
1064.                                    this->label1->Text = "0";
1065.                          }
1066.                          else {
1067.                                    this->label1->Text = currentText->Substring(1);
1068.                          }
1069.                }
1070.
1071.                else {
1072.                          this->label1->Text = "-" + currentText;
1073.                }
1074.     }
1075.     catch (Exception^ ex) {
1076.                this->label1->Text = "Error";
1077.     }
1078. }
1079. private: System::Void button14_Click(System::Object^ sender, System::EventArgs^ e) {
1080.     try {
1081.                String^ currentText = this->label1->Text;
1082.
1083.
1084.                if (currentText == "" || currentText == "0") {
1085.                          return;
1086.                }
1087.
1088.
1089.                currentText = currentText->Trim();
1090.
```

```
1091.
1092.                 double value = System::Convert::ToDouble(currentText);
1093.
1094.
1095.                 value = value / 100.0;
1096.
1097.
1098.                 if (Math::Abs(value) < 0.000001 && value != 0) {
1099.
1100.                         this->label1->Text = value.ToString("E6");
1101.                 }
1102.                 else if (Math::Abs(value) < 1.0) {
1103.
1104.                         this->label1->Text = value.ToString("0.##########");
1105.                 }
1106.                 else {
1107.
1108.                         this->label1->Text = value.ToString();
1109.                 }
1110.
1111.                 isNewOperation = true;
1112.     }
1113.     catch (FormatException^) {
1114.                 this->label1->Text = "Error: не число";
1115.     }
1116.     catch (OverflowException^) {
1117.                 this->label1->Text = "Error: слишком большое";
1118.     }
1119.     catch (Exception^) {
1120.                 this->label1->Text = "Error";
1121.     }
1122. }
1123.
1124.
1125. private: System::Void button18_Click(System::Object^ sender, System::EventArgs^ e) {
1126.     if (this->label1->Text != "" && this->label1->Text != "Error") {
1127.                 firstOperand = SafeConvert(this->label1->Text);
1128.                 currentOperation = "+";
1129.                 isNewOperation = true;
1130.                 this->label1->Text = "0";
1131.     }
1132. }
1133.
1134.
1135. private: System::Void button17_Click(System::Object^ sender, System::EventArgs^ e) {
1136.     if (this->label1->Text != "" && this->label1->Text != "Error") {
1137.                 firstOperand = SafeConvert(this->label1->Text);
1138.                 currentOperation = "-";
1139.                 isNewOperation = true;
1140.                 this->label1->Text = "0";
1141.     }
1142. }
1143.
1144.
1145. private: System::Void button16_Click(System::Object^ sender, System::EventArgs^ e) {
1146.     if (this->label1->Text != "" && this->label1->Text != "Error") {
1147.                 firstOperand = SafeConvert(this->label1->Text);
1148.                 currentOperation = "*";
1149.                 isNewOperation = true;
1150.                 this->label1->Text = "0";
1151.     }
1152. }
1153.
1154.
1155. private: System::Void button15_Click(System::Object^ sender, System::EventArgs^ e) {
1156.     if (this->label1->Text != "" && this->label1->Text != "Error") {
1157.                 firstOperand = SafeConvert(this->label1->Text);
1158.                 currentOperation = "/";
1159.                 isNewOperation = true;
1160.                 this->label1->Text = "0";
1161.     }
1162. }
1163.
1164.
1165. private: System::Void button19_Click(System::Object^ sender, System::EventArgs^ e) {
1166.     if (this->label1->Text != "" && this->label1->Text != "Error") {
```

```
1167.                 try {
1168.                         firstOperand = ParseMyNumber(this->label1->Text);
1169.                         currentOperation = "^";
1170.                         isNewOperation = true;
1171.                         this->label1->Text = "0";
1172.                 }
1173.                 catch (Exception^) {
1174.                         this->label1->Text = "Error";
1175.                 }
1176.     }
1177. }
1178.
1179.
1180. private: System::Void button21_Click(System::Object^ sender, System::EventArgs^ e) {
1181.
1182.     if (currentOperation == "") {
1183.                 return;
1184.     }
1185.
1186.     if (this->label1->Text != "" && this->label1->Text != "Error") {
1187.                 try {
1188.                         double secondOperand = SafeConvert(this->label1->Text);
1189.                         double result = 0;
1190.
1191.                         if (currentOperation == "+") {
1192.                                 result = firstOperand + secondOperand;
1193.                         }
1194.                         else if (currentOperation == "-") {
1195.                                 result = firstOperand - secondOperand;
1196.                         }
1197.                         else if (currentOperation == "*") {
1198.                                 result = firstOperand * secondOperand;
1199.                         }
1200.                         else if (currentOperation == "/") {
1201.                                 if (secondOperand == 0) {
1202.                                         this->label1->Text = "Error: /0";
1203.                                         currentOperation = "";
1204.                                         isNewOperation = true;
1205.                                         return;
1206.                                 }
1207.                                 result = firstOperand / secondOperand;
1208.                         }
1209.                         else if (currentOperation == "^") {
1210.                                 result = Math::Pow(firstOperand, secondOperand);
1211.                                 if (Double::IsInfinity(result) || Double::IsNaN(result)) {
1212.                                         this->label1->Text = "Error";
1213.                                         currentOperation = "";
1214.                                         isNewOperation = true;
1215.                                         return;
1216.                                 }
1217.                         }
1218.
1219.
1220.                         this->label1->Text = result.ToString();
1221.
1222.
1223.                         currentOperation = "";
1224.                         isNewOperation = true;
1225.                 }
1226.                 catch (...) {
1227.                         this->label1->Text = "Error";
1228.                         currentOperation = "";
1229.                         isNewOperation = true;
1230.                 }
1231.     }
1232. }
1233.         private: double ParseMyNumber(String^ text) {
1234.                 text = text->Trim();
1235.
1236.
1237.                 if (text == "") return 0;
1238.
1239.
1240.                 if (text == "-") return 0;
1241.
1242.
```

```
1243.                 bool hasComma = text->Contains(",");
1244.                 bool hasDot = text->Contains(".");
1245.
1246.                 if (hasComma && hasDot) {
1247.
1248.                         throw gcnew FormatException("Неверный формат числа");
1249.                 }
1250.
1251.
1252.                 if (hasComma) {
1253.                         text = text->Replace(",", ".");
1254.                 }
1255.
1256.
1257.                 for (int i = 0; i < text->Length; i++) {
1258.                         Char c = text[i];
1259.                         if (!(Char::IsDigit(c) || c == '.' || c == '-' || (c == 'E' || c == 'e'))) {
1260.                                 throw gcnew FormatException("Недопустимый символ: " + c);
1261.                         }
1262.                         if (c == '-' && i != 0) {
1263.                                 throw gcnew FormatException("Минус не в начале");
1264.                         }
1265.                 }
1266.
1267.
1268.                 return Double::Parse(text,
1269.                         System::Globalization::CultureInfo::InvariantCulture);
1270.         }
1271.
1272.
1273. private: System::Void button20_Click(System::Object^ sender, System::EventArgs^ e) {
1274.     try {
1275.                 String^ currentText = this->label1->Text;
1276.
1277.
1278.                 if (currentText == "" || currentText == "Error") {
1279.                         return;
1280.                 }
1281.
1282.
1283.                 currentText = currentText->Replace(",", ".");
1284.                 double value = Double::Parse(currentText,
1285.                         System::Globalization::CultureInfo::InvariantCulture);
1286.
1287.
1288.                 if (value < 0) {
1289.                         this->label1->Text = "Error: корень из отрицательного чиисла";
1290.                         return;
1291.                 }
1292.
1293.
1294.                 double result = Math::Sqrt(value);
1295.
1296.
1297.                 if (Math::Abs(result - Math::Round(result)) < 0.0000001) {
1298.
1299.                         this->label1->Text = Math::Round(result).ToString();
1300.                 }
1301.                 else {
1302.
1303.                         this->label1->Text = result.ToString("0.##########");
1304.                 }
1305.
1306.     }
1307.     catch (FormatException^) {
1308.                 this->label1->Text = "Error: не число";
1309.     }
1310.     catch (Exception^) {
1311.                 this->label1->Text = "Error";
1312.     }
1313. }
1314.
1315. private: System::Void button22_Click(System::Object^ sender, System::EventArgs^ e) {
1316.     this->label1->Text = (this->label1->Text == "Оконное приложение \"Калькулятор\"") ? "0" : "Оконное
приложение \"Калькулятор\"";
1317. }
```

```
1318. private: System::Void button24_Click(System::Object^ sender, System::EventArgs^ e) {
1319.     this->label1->Text = (this->label1->Text == "Автор приложения \"Кривнюк Диана\" 4К9392") ?
1320.                "0" :
1321.                "Автор приложения \"Кривнюк Диана\" 4К9392";
1322. }
1323.
1324. private: System::Void toolStripComboBox1_Click(System::Object^ sender, System::EventArgs^ e) {
1325. }
1326. };
1327.
1328.
1329. }
1330.
```

Код библиотеки(cpp):

```
 1. #include <windows.h>
 2. #include <string.h>
 3. #include "pch.h"
 4.
 5. extern "C" __declspec(dllexport) char* ProcessKey(int key)
 6. {
 7.     // Выделяем память через LocalAlloc (работает в Windows)
 8.     char* result = (char*)LocalAlloc(LPTR, 20);
 9.     if (result == NULL) return NULL;
10.
11.     // Обработка цифр (основная клавиатура - коды 48-57)
12.     if (key >= 48 && key <= 57) {
13.         result[0] = (char)key;
14.         result[1] = 0;
15.     }
16.     // Обработка цифр (NumPad - коды 96-105)
17.     else if (key >= 96 && key <= 105) {
18.         result[0] = (char)('0' + (key - 96));
19.         result[1] = 0;
20.     }
21.     // Операции
22.     else if (key == 107 || key == 187) { // VK_ADD (107) или OEMPLUS (187)
23.         strcpy_s(result, 20, "plus");
24.     }
25.     else if (key == 109 || key == 189) { // VK_SUBTRACT (109) или OEMINUS (189)
26.         strcpy_s(result, 20, "minus");
27.     }
28.     else if (key == 106) { // VK_MULTIPLY (106)
29.         strcpy_s(result, 20, "mul");
30.     }
31.     else if (key == 111) { // VK_DIVIDE (111)
32.         strcpy_s(result, 20, "div");
33.     }
34.     else if (key == 13) { // VK_RETURN (13)
35.         strcpy_s(result, 20, "=");
36.     }
37.     else {
38.         strcpy_s(result, 20, "none");
39.     }
40.
41.     return result;
42. }
43.
```

Контрольные вопросы:

1. Дайте краткое понятие главного меню и его роль в пользовательском интерфейсе приложения на C++.

Главное меню — это основной элемент графического интерфейса, представляющий собой горизонтальную полосу с раскрывающимися списками команд (Файл, Правка, Вид и т.д.). Его роль заключается в обеспечении структурированного

доступа ко всем основным функциям приложения, предоставлении пользователю понятной иерархии действий без загромождения рабочей области.

2. Раскройте значение контекстного меню и его использование для улучшения взаимодействия пользователя с программой.

Контекстное меню — это всплывающее меню, которое появляется по щелчку правой кнопкой мыши и содержит команды, относящиеся только к текущему выделенному объекту или контексту (например, «Копировать», «Вставить», «Свойства» для выделенного текста). Его значение в ускорении работы и интуитивности: пользователь получает быстрый доступ к релевантным именно сейчас действиям, не отвлекаясь на поиск нужной команды в главном меню.

3. Сформулируйте сущность библиотечных модулей пользователя и их применение в разработке программ на C++.

Библиотечные модули пользователя — это самодельные заголовочные файлы (.h) и файлы исходного кода (.cpp), которые содержат часто используемые функции, классы или константы, объединенные по какому-либо признаку. Их применение позволяет структурировать код, избегать его дублирования, упрощать отладку и повторно использовать написанный код в разных проектах, подключая их директивой #include "имя_файла.h".

4. Изложите особенности работы с внешними файлами в программах на C++ и как они могут использоваться для хранения данных.

Работа с внешними файлами в C++ осуществляется через библиотеки (<fstream>), предоставляющие классы ifstream (чтение), ofstream (запись) и fstream (чтение/запись). Особенность — данные не привязаны к времени жизни программы: они сохраняются на диске после её закрытия. Это позволяет использовать файлы для долговременного хранения настроек пользователя, результатов вычислений, баз данных (в простом текстовом виде), логов работы программы или больших объемов входных данных.

5. Раскройте особенности разработки алгоритмов для решения задач с использованием внешних файлов на C++.

При разработке алгоритмов для работы с файлами ключевой особенностью становится необходимость контроля состояния потока (успешно ли открылся файл, не достигнут ли конец файла eof()). Алгоритмы должны обрабатывать ситуации отсутствия файла, ошибок чтения/записи и правильно управлять позицией чтения/записи в файле (особенно при бинарном доступе). Важно также своевременно закрывать файл (close()) для освобождения ресурсов.

6. Дайте определение дополнительным элементам управления в диалоговых окнах и их роль в создании интерактивного интерфейса.

Дополнительные элементы управления — это стандартные компоненты интерфейса, размещаемые в диалоговых окнах: кнопки, текстовые поля, списки,

переключатели, флажки, ползунки. Их роль — сбор и ввод пользовательских данных, настройка параметров операции и управление логикой приложения через визуальное взаимодействие, делая программу более гибкой и удобной.

7. Сформулируйте особенности обработки событий в главном и контекстном меню на языке C++ и как это влияет на работу программы.

Особенность обработки событий меню (обычно это событие ON_COMMAND или Click) заключается в том, что оно построено на принципе "один пункт — одно действие". Каждому пункту меню сопоставляется уникальный идентификатор (ID), при выборе которого генерируется событие, вызывающее конкретную функцию-обработчик. Это напрямую влияет на работу программы, так как обеспечивает модульность: код для каждой команды изолирован, что упрощает добавление новых функций и изменение логики существующих без затрагивания других частей интерфейса.